ZERO TO SENIOR ENGINEER · LAUGHTALE.CO.ZA

DSA MASTERY

QUICK REFERENCE

7	33	3
Domains	Concepts	Focus Areas

01 · COMPLEXITY FOUNDATIONS
02 · LINEAR DATA STRUCTURES
03 · RECURSION & BINARY SEARCH
04 · TREES & HEAPS
05 · SORTING ALGORITHMS
06 · GRAPH THEORY
07 · ALGORITHMIC PATTERNS

- Each row: TOPIC → MECHANISM → WHY/WHEN IT MATTERS → KEY TERMS
- Gold rows = classically hardest topics — expect to spend more repetitions here

Print: Chrome/Edge → Print → A4 · Background graphics ON · Landscape

DOMAIN 01

COMPLEXITY FOUNDATIONS

The language every other DSA topic is measured against.

4 CONCEPTS

0 FOCUS

TOPIC	MECHANISM	WHY / WHEN IT MATTERS	KEY TERMS
BIG O NOTATION	Describes how runtime and memory scale as input size *n* grows, capturing only the dominant term while dropping constants — `O(3n+7)` simplifies to `O(n)`. It's typically a worst-case bound, giving a hardware-independent way to reason about scalability before code is even written.	This is the language every technical interview and system design conversation uses to compare approaches, so fluency here is non-negotiable before any other DSA topic makes sense. Master reading nested loops (`O(n²)`), halving patterns (`O(log n)`), and recursive branching (`O(2^n)`) on sight.	*O(1) O(log n) O(n) O(n log n) O(n²)*
STATIC ARRAYS	A fixed-size, contiguous block of memory allocated once at creation — indexing is `O(1)` because the address of any element is computed directly from the base address and index. Resizing isn't possible; if you need more space you must allocate an entirely new block.	Understand static arrays as the conceptual foundation beneath every other structure — languages like C and Java expose them directly, and their fixed-size constraint explains why dynamic arrays exist at all. In-place array manipulation questions assume this fixed-memory mental model.	*contiguous memory, fixed size, O(1) index*
DYNAMIC ARRAYS	Wraps a static array but automatically reallocates to a larger block (usually doubling capacity) when full, copying existing elements over. This amortized doubling is what makes `.append()` `O(1)` on average, even though any single resize operation is `O(n)`.	Nearly every array-based problem uses a dynamic array under the hood, so understanding amortized cost explains why appending is "usually free" but occasionally expensive. It's also why inserting at the front is always `O(n)` — every element must shift regardless of resizing.	*amortized O(1), resizing, doubling*
STRINGS	Behave like arrays of characters, but are immutable in Python — every concatenation creates an entirely new string object rather than mutating in place, making repeated concatenation in a loop `O(n²)` overall. Building via a list and joining at the end avoids this trap.	This immutability trap is one of the most common hidden performance bugs in interview code — recognizing "am I concatenating in a loop" and switching to `list` + `join()` is a signal of real fluency. Slicing is `O(k)` for a slice of length k, not `O(1)`, which matters for substring-heavy algorithms.	*immutability, join(), O(n²) concat trap*

DOMAIN 02

LINEAR DATA STRUCTURES

Pointer-based and hash-based structures that trade array-shifting cost for something faster.

6 CONCEPTS

0 FOCUS

TOPIC	MECHANISM	WHY / WHEN IT MATTERS	KEY TERMS
SINGLY LINKED LISTS	A chain of nodes, each storing a value and a pointer to the next node — no contiguous memory, so insertion/deletion at a known node is `O(1)` since it's just pointer rewiring, not shifting. The tradeoff is `O(n)` access to an arbitrary index, since you must walk the chain from the head.	Use when you need frequent insertion/deletion at the head or middle without the `O(n)` shifting cost a dynamic array pays. This structure is also the backbone for understanding more complex pointer-based structures like trees and graphs.	*node, pointer, head, O(1) insert*
DOUBLY LINKED LISTS	Adds a "previous" pointer alongside "next," letting you traverse both directions and delete a node in `O(1)` once you have a reference to it, without walking from the head to find its predecessor. The cost is extra memory per node and more bookkeeping to keep both directions consistent.	Reach for this when you need `O(1)` removal from either end (like an LRU cache) or need to walk backward — Python's `collections.deque` is implemented this way, giving `O(1)` appends/pops from both ends.	*prev/next pointer, deque, O(1) delete*
HASH TABLES & HASH FUNCTIONS	Stores key-value pairs by running each key through a hash function to compute an index (bucket), giving `O(1)` average lookup, insert, and delete regardless of table size. Collisions (two keys hashing to the same bucket) are resolved via chaining or open addressing, and the table resizes once it gets too full.	This is the single most powerful lookup structure in DSA — "I need to check membership or count occurrences fast" should immediately trigger "hash table." A poor hash function degrades performance to `O(n)`, so the mechanism explains both the power and the failure mode.	*hash function, collision, chaining, load factor*
SETS & MAPS	A set is a hash table storing only keys, optimized purely for `O(1)` membership testing and automatic deduplication. A map (Python `dict`) is a hash table storing key-value pairs, letting you associate arbitrary data with each unique key at the same average-case cost.	Use a set the moment you need "has this been seen before" without extra data — deduplicating, tracking visited nodes. Use a map to associate a count, index, or object with a key, like frequency counting or memoization caches in dynamic programming.	*dedup, membership test, frequency map*
STACKS	A Last-In-First-Out (LIFO) structure supporting `O(1)` push and pop, both restricted to one end — typically built on a dynamic array or linked list, since both give `O(1)` at that single end. No direct access to the middle; only the top is ever visible.	Recognize a stack the moment a problem involves "undo the most recent thing," matching nested pairs (parentheses, brackets), or unwinding history in reverse order. It's also the literal structure behind recursive call stacks and depth-first search.	*LIFO, push/pop, matching brackets*
QUEUES	A First-In-First-Out (FIFO) structure supporting `O(1)` enqueue at the back and `O(1)` dequeue from the front — built with a doubly linked list or circular buffer, since a plain dynamic array would pay `O(n)` to remove from the front. `collections.deque` guarantees `O(1)` at both ends.	Recognize a queue whenever a problem must process items in the exact order they arrived — breadth-first search, task scheduling, buffering streams. Using a plain list with `.pop(0)` as a queue is a common bug, since that call is actually `O(n)`.	*FIFO, enqueue/dequeue, deque*

DOMAIN 03

RECURSION & BINARY SEARCH

Solving problems by shrinking them — either into smaller identical subproblems, or into a halved search space.

3 CONCEPTS

1 FOCUS

TOPIC	MECHANISM	WHY / WHEN IT MATTERS	KEY TERMS
RECURSION & CALL STACKS	A recursive function calls itself with a smaller version of the problem until it hits a base case, and Python manages this with an actual call stack — each call pushes a new stack frame holding its local variables, and returning pops it off. Every recursive algorithm has an equivalent iterative version using an explicit stack; recursion just lets the language manage that stack for you.	Recursion is the natural fit whenever a problem can be defined in terms of smaller identical subproblems — tree traversal, divide and conquer, backtracking. Understanding the call stack explains both why deep recursion can hit a `RecursionError` and why every recursive solution can be rewritten iteratively.	*base case, stack frame, recursion depth*
BINARY SEARCH (TRADITIONAL)	Repeatedly halves a sorted search space, comparing the middle element to the target and discarding the half that can't contain it, giving `O(log n)` instead of `O(n)` linear scanning. Tracks `left` and `right` pointers and a `mid = (left + right) // 2`, updating boundaries until found or the space is empty.	The precondition is always "the data must be sorted" — applying it to unsorted data silently gives wrong answers, not an error. Recognize it the moment a problem says "sorted array" and asks for a specific value or insertion point.	*sorted array, O(log n), left/right pointers*
BINARY SEARCH (CONDITION-BASED) ⚠ FOCUS	Applies the same halving logic to a range of possible answers rather than an array — guess a value, check whether it satisfies a monotonic condition (true for all values on one side, false on the other), and binary search on that boolean boundary instead of a literal sorted list. Works whenever the problem has a monotonic yes/no structure, even without an explicit sorted array in sight.	This is the harder, less obvious form of binary search that trips people up in interviews — recognizing "can I binary search the answer space instead of the input" turns an `O(n)` or worse brute force into `O(log n)`. Look for phrasing like "minimum value such that X is possible."	*search space, monotonic condition, "search on answer"*

DOMAIN 04

TREES & HEAPS

Hierarchical structures — from general shape, to ordering invariants, to priority guarantees.

4 CONCEPTS

0 FOCUS

TOPIC	MECHANISM	WHY / WHEN IT MATTERS	KEY TERMS
BINARY TREES	A hierarchical structure where each node has at most two children (left/right), with no ordering guarantee between them — the general structural shape underlying BSTs, heaps, and expression trees. Traversal order (preorder, inorder, postorder, level-order) determines the sequence nodes are visited in, chosen based on what the algorithm needs.	Master the recursive pattern: process the current node, then recurse on left and right subtrees — nearly every tree algorithm is a variation of this template. Level-order traversal (via a queue) is what you reach for when the problem cares about depth or "level by level."	*node, left/right child, traversal order*
BINARY SEARCH TREES (BST)	Adds an ordering invariant to a binary tree: every node's left subtree contains only smaller values, its right only larger ones — an inorder traversal produces sorted output for free. Search, insert, delete are all `O(log n)` on a balanced tree, but degrade to `O(n)` if the tree becomes skewed from inserting already-sorted data.	Reach for a BST when you need sorted-order data with fast insert/search, but be aware an unbalanced BST loses its performance guarantee entirely — exactly why self-balancing variants (AVL, Red-Black trees) exist in production systems.	*ordering invariant, O(log n), skewed tree risk*
HEAPS	A complete binary tree (filled left to right, level by level) satisfying the heap property — in a min-heap, every parent is smaller than its children, so the smallest element sits at the root, retrievable in `O(1)`. Heapify (sift-up on insert, sift-down on removal) restores the property in `O(log n)`, usually stored compactly in an array using index math instead of pointers.	Reach for a heap whenever you repeatedly need "the current minimum/maximum" as data streams in — heap sort builds a heap in `O(n)` then extracts the root repeatedly in `O(log n)` for overall `O(n log n)`. The structure behind every "top-k" interview problem.	*heap property, sift-up/down, heap sort*
PRIORITY QUEUES	The abstract concept — "dequeue always returns the highest-priority item" — with a heap as the most common concrete implementation, giving `O(log n)` insert and removal. Python's `heapq` implements a min-heap directly on top of a plain list, so `heapq.heappush` and `heapq.heappop` are the two core operations.	Recognize a priority queue whenever a problem needs "process items in order of importance, not arrival order" — Dijkstra's shortest path, deadline scheduling, merging k sorted lists. Remember `heapq` is min-heap only — negate values for max-heap behavior.	*heapq, heappush/heappop, min-heap default*

DOMAIN 05

SORTING ALGORITHMS

Six ways to order a sequence — each with a different tradeoff between time, space, and stability.

6 CONCEPTS

0 FOCUS

TOPIC	MECHANISM	WHY / WHEN IT MATTERS	KEY TERMS
BUBBLE SORT	Repeatedly steps through the list, swapping adjacent out-of-order elements, so after each pass the largest unsorted element "bubbles" to its correct position — $O(n^2)$ comparisons and swaps worst/average case. An early-exit flag makes it $O(n)$ best case on already-sorted input.	Rarely used in production due to $O(n^2)$ cost, but the canonical first example for understanding comparison-based sorting and swap mechanics before faster algorithms. Sometimes used to test whether you can identify why a naive approach is inefficient.	*adjacent swap, $O(n^2)$, early exit*
INSERTION SORT	Builds the sorted portion one element at a time, shifting each new element backward through the sorted prefix until it lands in place — $O(n^2)$ worst case, but only $O(n)$ on nearly-sorted data since few shifts are needed. The same mental model as sorting playing cards by hand.	Reach for this specifically when data is nearly sorted already, or the dataset is small — its low constant-factor overhead beats $O(n \log n)$ algorithms below a size threshold, which is why production sorts switch to it for small sub-arrays.	*shifting, nearly-sorted, $O(n)$ best case*
SELECTION SORT	Repeatedly scans the unsorted portion to find the minimum, then swaps it into place at the front of the unsorted section — always $O(n^2)$ comparisons regardless of input order, since a full scan happens every pass. Its advantage is a minimal number of actual swaps (at most n).	Rarely the right choice given its fixed $O(n^2)$ cost, but builds the "find minimum, swap into place" pattern used elsewhere. Useful specifically when swap cost dominates and you want to minimize total writes to memory.	*minimum scan, fixed $O(n^2)$, few swaps*
MERGE SORT	A divide-and-conquer algorithm that recursively splits the list in half until each piece has one element, then merges sorted halves by comparing their fronts — guarantees $O(n \log n)$ in every case since split and merge steps are fixed regardless of input order. Costs $O(n)$ extra space, since the merge isn't in-place.	Reach for this when you need guaranteed $O(n \log n)$ regardless of input, or stability (equal elements keep relative order) matters — the standard choice for sorting linked lists, where sequential access fits naturally.	*divide & conquer, stable, $O(n)$ space*
QUICK SORT	Picks a pivot, partitions the list so smaller elements go left and larger go right, then recursively sorts each side — average $O(n \log n)$, but worst case $O(n^2)$ if pivot choice repeatedly produces badly unbalanced partitions. Randomized or median-of-three pivot selection avoids this in practice.	Generally the fastest general-purpose sort in practice due to cache locality and in-place partitioning ($O(\log n)$ space, not $O(n)$ like merge sort) — why most standard libraries' default sort is quicksort-based. Pivot-selection risk is the classic interview follow-up.	*pivot, partition, in-place, worst-case $O(n^2)$*
COUNTING SORT	Instead of comparing elements, tallies how many times each distinct value appears in a count array indexed by value, then reconstructs the sorted output directly from those counts — achieves $O(n + k)$ time, sidestepping the $O(n \log n)$ comparison-sort lower bound. Only works for integers within a known, reasonably small range.	Reach for this sorting a large number of items when the value range is small and known in advance — exam scores 0-100, characters in a string — since it beats comparison sorts asymptotically in that scenario. Breaks down for large or unbounded ranges.	*non-comparison, $O(n+k)$, bounded range*

DOMAIN 06

GRAPH THEORY

Representing networks, then traversing them by depth or by level.

5 CONCEPTS

0 FOCUS

TOPIC	MECHANISM	WHY / WHEN IT MATTERS	KEY TERMS
EDGE LIST	The simplest graph representation — a flat list of (source, destination) pairs, optionally weighted — taking O(E) space. Compact and easy to build, but checking "are these two nodes connected" requires scanning the whole list.	Use when you mainly need to iterate all edges once (like sorting edges by weight for Kruskal's MST algorithm), rather than repeated neighbor lookups. Rarely the representation you traverse a graph with directly — usually converted to an adjacency list first.	*(u, v) pairs, O(E) space, Kruskal's*
ADJACENCY MATRIX	A 2D grid of size V×V where cell [i][j] indicates an edge between node i and j (or its weight) — checking if two nodes are connected is O(1), a direct index lookup. Costs O(V²) space regardless of how many edges actually exist.	Reach for this when the graph is dense (edges close to V²) or O(1) "are these two nodes directly connected" lookups dominate, like in some DP-over-graph formulations. Avoid for large sparse graphs — memory cost becomes prohibitive fast.	*V×V grid, O(1) edge check, O(V²) space*
ADJACENCY LIST	Each node keeps a list of its direct neighbors, giving O(V + E) total space — far more efficient than a matrix for sparse graphs, since only existing edges are stored. Checking two nodes' connection is O(degree), but iterating all neighbors of a node is fast and direct.	The default, go-to representation for almost every graph problem, since most real-world graphs are sparse and most algorithms (DFS, BFS, Dijkstra) fundamentally need "give me this node's neighbors." Reach for a matrix only with a specific dense-graph or O(1)-lookup requirement.	*O(V+E) space, neighbor lookup, default choice*
DEPTH-FIRST SEARCH (DFS)	Explores as far down one path as possible before backtracking, implemented with recursion (call stack) or an explicit stack — visit a node, mark visited, recurse into each unvisited neighbor before returning. O(V + E) since every node and edge is examined once.	Reach for DFS when you need to explore all paths, detect cycles, perform topological sorting, or solve connectivity problems where exploration order matters less than covering everything reachable. Overlaps heavily with backtracking.	*stack-based, backtracking, O(V+E)*
BREADTH-FIRST SEARCH (BFS)	Explores a graph level by level, visiting all of a node's direct neighbors before moving to theirs, implemented with a queue — dequeue a node, mark visited, enqueue unvisited neighbors, repeat. Also O(V + E), but the queue-based level order is what distinguishes it from DFS's stack-based depth-first order.	Reach for BFS whenever you need shortest path in an unweighted graph — the level-by-level guarantee means the first time you reach a node is via the shortest path. The algorithm behind shortest-path problems, level-order traversal, and "minimum steps" problems.	*queue-based, shortest path (unweighted), level order*

DOMAIN 07

ALGORITHMIC PATTERNS & ADVANCED TECHNIQUES

The reusable playbook that turns brute-force solutions into interview-ready ones.

5 CONCEPTS

2 FOCUS

TOPIC	MECHANISM	WHY / WHEN IT MATTERS	KEY TERMS
TWO POINTERS	Uses two index variables moving through a structure (often a sorted array) — converging from opposite ends, or both moving forward at different speeds — to solve problems in a single $O(n)$ pass instead of the $O(n^2)$ a naive nested loop would need. Relies on a structural property (usually sortedness) guaranteeing that moving one pointer, rather than checking every pair, is safe.	Recognize this whenever a brute-force solution checks all pairs in a sorted array or sequence — pair-sum problems, palindrome checks, removing duplicates in place. One of the highest-leverage patterns in interviews, turning an obvious $O(n^2)$ into $O(n)$.	*converging pointers, O(n) pass, sorted input*
SLIDING WINDOW	Maintains a contiguous subrange defined by two pointers that expands and contracts over a sequence, avoiding recomputing a full subrange sum/count from scratch each move. A fixed-length window has a constant size k that slides one step at a time; a variable-length window grows via the right pointer and shrinks via the left pointer whenever a condition is violated.	Recognize this whenever a problem involves a contiguous subarray/substring and a target condition — "longest substring without repeating characters," "smallest subarray with sum ≥ target." Turns an $O(n^2)$ or $O(n \cdot k)$ brute-force rescan into a single $O(n)$ pass.	*expand/contract, fixed vs variable, O(n) pass*
RECURSIVE BACKTRACKING ⚠ FOCUS	Builds a solution incrementally, and the moment a partial solution can't possibly lead to a valid complete one, abandons ("backtracks") that branch and tries the next option — recursion with an explicit "undo" step after each call returns, restoring shared state before the next choice. Systematically explores the entire solution space while pruning invalid branches early.	Recognize this whenever a problem asks for "all possible" combinations, permutations, subsets, or valid arrangements (N-Queens, Sudoku, generating parentheses) — anywhere brute force must explore a tree of choices but can cut branches short once clearly invalid.	*choose-explore-unchoose, pruning, exponential search space*
DYNAMIC PROGRAMMING ⚠ FOCUS	Solves problems with overlapping subproblems and optimal substructure by storing each subproblem's result once computed, avoiding exponential recomputation. Top-down memoization keeps the natural recursive structure but adds a cache checked before recursing further; bottom-up tabulation builds an iterative table from the smallest subproblems upward, trading recursion overhead for a small performance edge and no stack-overflow risk.	Reach for DP the moment a recursive brute-force solution recomputes identical subproblems repeatedly — the signal is an exponential recursion tree with heavily overlapping branches (Fibonacci, knapsack, edit distance). Start with top-down memoization since it mirrors the natural recursive definition, then convert to bottom-up once the recurrence is clear.	*memoization, tabulation, optimal substructure*
BINARY NUMBERS & BIT MANIPULATION	Numbers are stored in binary under the hood, and bitwise operators (`&` `|` `^` `~` `<<` `>>`) let you manipulate individual bits directly — e.g. `n & (n-1)` clears the lowest set bit, `n << 1` multiplies by 2. These run in true $O(1)$ constant time at the hardware level, regardless of the number's magnitude.	Reach for bit manipulation when a problem involves flags, subsets (each bit = "included" or not), or asks for extreme performance/memory efficiency — counting set bits, finding a single non-duplicate via XOR, generating all subsets via bitmasking. Also used to compress boolean flags into one integer.	*AND/OR/XOR, bitmask, O(1) ops*